Manual 3: Running R and Python Scripts on CESNET Kubernetes

Pollination Ecology Group (Handrkov) — CPU & GPU Batch Jobs Without Building Docker Images

2026-11-12

Table of contents

1. The Fastest Way: One-Command Launcher (`run_script_on_k8s.sh`) 2
2. Step-by-Step: Running an R Script on Kubernetes (`r-cpu-job.yaml`) 2
   - Step 1: Write Your R Script (`my_analysis.R`) 2
   - Step 2: Upload the Script as a `ConfigMap` 3
   - Step 3: Submit the Job & Watch the Logs 3
3. Step-by-Step: Running a Python Script on CPU (`python-cpu-job.yaml`) 4
   - Step 1: Write Your Python Script (`process_data.py`) 4
   - Step 2: Upload the `ConfigMap` and Submit `templates/k8s/python-cpu-job.yaml` . . . 5
4. Step-by-Step: Running a GPU Job (YOLO / PyTorch Deep Learning) 5
   - Three Golden Rules for GPU Jobs on CERIT-SC: 5
   - Submitting a GPU Job 6
5. Interactive Debugging Pod (Testing Commands Live in a Terminal) 6
6. Housekeeping: Cleaning Up Finished Jobs 7

One of the biggest misconceptions about Kubernetes is that you must build and push a new Docker image every time you edit a line of R or Python code.

In our group's workflow, **you do NOT need to build Docker images for 95% of scripts**. Instead, we use:

1. A standard pre-built scientific container image (`rocker/tidyverse:4.4.1` for R, `python:3.12-slim` for Python, or `ultralytics/ultralytics:latest` for PyTorch/YOLO GPU jobs).
2. A **Kubernetes `ConfigMap`** that injects your `.R` or `.py` script directly into `/scripts/` inside the container at startup.
3. The **`s3-credentials Secret`** in `stenc-ns` so your script can automatically download input datasets from `peg-bucket` (`https://s3.cl4.du.cesnet.cz`) and upload its results back when finished.

---

1. The Fastest Way: One-Command Launcher (`run_script_on_k8s.sh`)

If you already have an `.R` or `.py` script that reads its inputs from CESNET S3 and writes its outputs back to CESNET S3, you can run it on the cluster with **a single terminal command** using `templates/k8s/run_script_on_k8s.sh`:

```
# Run an R script on 8 CPU cores & 16 GB RAM (default):
./templates/k8s/run_script_on_k8s.sh my_analysis.R

# Run a heavy R script requesting 16 CPU cores & 32 GB RAM:
./templates/k8s/run_script_on_k8s.sh heavy_simulation.R --cpu 16 --mem 32Gi

# Run a Python script on an NVIDIA GPU:
./templates/k8s/run_script_on_k8s.sh train_yolo.py --gpu
```

What `run_script_on_k8s.sh` does behind the scenes:

1. Packages your local `.R` or `.py` file into a Kubernetes `ConfigMap`.
2. Creates a CERIT-SC security-compliant `Job` with `s3-credentials` environment variables (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `S3_ENDPOINT`, `S3_BUCKET=peg-bucket`).
3. Waits for the pod to start and streams the live output logs to your terminal. *(Note: Pressing `Ctrl+C` only stops log streaming on your laptop—the job keeps running on the cluster!)*

---

2. Step-by-Step: Running an R Script on Kubernetes (`r-cpu-job.yaml`)

If you want full control over R packages or multi-file scripts, follow this 3-step workflow using `templates/k8s/r-cpu-job.yaml`.

Step 1: Write Your R Script (`my_analysis.R`)

Because the Kubernetes pod starts with an empty local disk (`/app/workspace`), your R script should read input files from `peg-bucket` and upload output files back to `peg-bucket`:

```
# my_analysis.R
library(aws.s3)

S3_BUCKET <- Sys.getenv("S3_BUCKET", unset = "peg-bucket")
S3_HOST   <- "s3.cl4.du.cesnet.cz"

cat("1. Downloading input dataset from CESNET S3...\n")
save_object(
  object   = "Ostatni/MyProject/input_data.csv",
  bucket   = S3_BUCKET,
  file     = "input_data.csv",
  base_url = S3_HOST,
```

```
  region   = "",
  use_https = TRUE
)

df <- read.csv("input_data.csv")
cat("Loaded", nrow(df), "rows.\n")

# --- Run your statistical model / simulation here ---
summary_df <- aggregate(. ~ site, data = df, FUN = mean)
write.csv(summary_df, "summary_results.csv", row.names = FALSE)
saveRDS(summary_df, "summary_results.rds")

cat("2. Uploading results back to CESNET S3...\n")
put_object(
  file     = "summary_results.csv",
  object   = "Ostatni/MyProject/outputs/summary_results.csv",
  bucket   = S3_BUCKET,
  base_url = S3_HOST,
  region   = "",
  use_https = TRUE
)
put_object(
  file     = "summary_results.rds",
  object   = "Ostatni/MyProject/outputs/summary_results.rds",
  bucket   = S3_BUCKET,
  base_url = S3_HOST,
  region   = "",
  use_https = TRUE
)
cat("All done!\n")
```

Step 2: Upload the Script as a `ConfigMap`

Run this command in your terminal whenever you create or update `my_analysis.R`:

```
kubectl create configmap my-r-script -n stenc-ns \
  --from-file=my_analysis.R=my_analysis.R \
  --dry-run=client -o yaml | kubectl apply -f -
```

(You can also pass multiple `--from-file=` arguments if your R script calls `source("helper_functions.R")`!)

Step 3: Submit the Job & Watch the Logs

Copy `templates/k8s/r-cpu-job.yaml` to your project directory, adjust the list of CRAN packages (`pkgs <- c("aws.s3", "lme4", "brms", ...)`) or CPU/RAM limits if needed, and submit:

```
# Delete any old finished job with the same name first:
kubectl delete job r-analysis-job -n stenc-ns --ignore-not-found

# Start the job:
kubectl apply -f r-cpu-job.yaml

# Watch live R output:
kubectl logs -f job/r-analysis-job -n stenc-ns
```

> **Note**
>
> **How R Package Installation Works in Non-Root Pods:** Because CERIT-SC pods run as user 1000, R cannot write to `/usr/local/lib/R/site-library`. In `templates/k8s/r-cpu-job.yaml`, we set `R_LIBS_USER=/app/R_libs` (on the writable `/app` volume) so `install.packages()` works without root permissions!

---

3. Step-by-Step: Running a Python Script on CPU (`python-cpu-job.yaml`)

Step 1: Write Your Python Script (`process_data.py`)

```
# process_data.py
import os
import boto3
from botocore.config import Config
import pandas as pd

S3_ENDPOINT = os.environ["S3_ENDPOINT"]
S3_BUCKET   = os.environ["S3_BUCKET"]

s3 = boto3.client(
    "s3",
    endpoint_url=S3_ENDPOINT,
    aws_access_key_id=os.environ["AWS_ACCESS_KEY_ID"],
    aws_secret_access_key=os.environ["AWS_SECRET_ACCESS_KEY"],
    config=Config(signature_version="s3v4"),
)

print("1. Downloading input data from S3...")
s3.download_file(S3_BUCKET, "Ostatni/MyProject/raw_counts.csv", "raw_counts.csv")

df = pd.read_csv("raw_counts.csv")
print(f"Processing {len(df)} records...")

# --- Your computation here ---
```

```
df["normalized"] = df["count"] / df["count"].sum()
df.to_csv("processed_counts.csv", index=False)

print("2. Uploading results to S3...")
s3.upload_file("processed_counts.csv", S3_BUCKET, "Ostatni/MyProject/processed_counts.csv")
print("Done!")
```

Step 2: Upload the `ConfigMap` and Submit `templates/k8s/python-cpu-job.yaml`

```
# 1. Upload/update your Python script ConfigMap
kubectl create configmap my-python-script -n stenc-ns \
  --from-file=process_data.py=process_data.py \
  --dry-run=client -o yaml | kubectl apply -f -

# 2. Submit the Job
kubectl delete job python-cpu-job -n stenc-ns --ignore-not-found
kubectl apply -f templates/k8s/python-cpu-job.yaml

# 3. Follow live logs
kubectl logs -f job/python-cpu-job -n stenc-ns
```

---

4. Step-by-Step: Running a GPU Job (YOLO / PyTorch Deep Learning)

When training computer vision models (e.g., YOLO pollen detection, hoverfly tracking, stomata segmentation, root segmentation), use `templates/k8s/python-gpu-job.yaml`.

Three Golden Rules for GPU Jobs on CERIT-SC:

1. **Always Mount `/dev/shm` (Shared Memory):** By default, Kubernetes gives containers only `64 MB` of `/dev/shm`. PyTorch and Ultralytics YOLO `DataLoader` workers use `/dev/shm` to pass image batches between CPU and GPU; with only 64 MB, your training job will crash mid-epoch with `RuntimeError: DataLoader worker (pid ...) is killed by signal: Bus error`. Our template `templates/k8s/python-gpu-job.yaml` mounts a **4 GiB RAM-backed `/dev/shm` volume**:

```
    volumeMounts:
    - name: dshm
      mountPath: /dev/shm
  volumes:
  - name: dshm
    emptyDir:
      medium: Memory
      sizeLimit: 4Gi
```

2. **Stage Training Datasets as a Single `.zip` on S3 and Unpack into `/app/workspace`:** Never train a model directly over an `s3fs` mount! Instead, pack your training images and labels into a `.zip` archive on S3 (`s3://peg-bucket/Ostatni/MyProject/dataset.zip`), download it into `/app/workspace` at the start of the job using `aws s3 cp`, unzip it locally on the fast node SSD, run training, and `aws s3 sync` the `runs/` folder back to `peg-bucket` when finished.
3. **Set Writable Config Directories for Non-Root User `1000`:** Tools like Ultralytics YOLO and Matplotlib try to write config files to `~/.config`. Always export:

```
export HOME=/app
export YOLO_CONFIG_DIR=/app/.config/Ultralytics
```

Submitting a GPU Job

```
# 1. Upload your training script to the ConfigMap
kubectl create configmap my-gpu-script -n stenc-ns \
  --from-file=train.py=train.py \
  --dry-run=client -o yaml | kubectl apply -f -

# 2. Submit the GPU job
kubectl delete job python-gpu-job -n stenc-ns --ignore-not-found
kubectl apply -f templates/k8s/python-gpu-job.yaml

# 3. Monitor training epochs live
kubectl logs -f job/python-gpu-job -n stenc-ns
```

---

5. Interactive Debugging Pod (Testing Commands Live in a Terminal)

Sometimes you want to open an interactive shell on the cluster to test S3 paths, inspect files, or debug package installations before submitting a long batch job.

Run this command to launch a temporary interactive Python/S3 shell inside `stenc-ns` (it automatically deletes itself when you type `exit`):

```
kubectl run interactive-debug -n stenc-ns --rm -it --restart=Never \
  --image=python:3.12-slim \
  --overrides='{
    "spec": {
      "securityContext": {
        "runAsNonRoot": true,
        "runAsUser": 1000,
        "runAsGroup": 1000,
        "seccompProfile": {"type": "RuntimeDefault"}
      },
      "containers": [{
```

```
        "name": "debug",
        "image": "python:3.12-slim",
        "command": ["bash"],
        "stdin": true,
        "tty": true,
        "workingDir": "/app",
        "securityContext": {
          "allowPrivilegeEscalation": false,
          "capabilities": {"drop": ["ALL"]}
        },
        "env": [
          {"name": "HOME", "value": "/app"},
          {"name": "S3_ENDPOINT", "value": "https://s3.cl4.du.cesnet.cz"},
          {"name": "S3_BUCKET", "value": "peg-bucket"},
          {"name": "AWS_ACCESS_KEY_ID", "valueFrom": {"secretKeyRef": {"name": "s3-credentials",
          {"name": "AWS_SECRET_ACCESS_KEY", "valueFrom": {"secretKeyRef": {"name": "s3-credentia
        ],
        "volumeMounts": [{"name": "app-vol", "mountPath": "/app"}]
      }],
      "volumes": [{"name": "app-vol", "emptyDir": {}}]
    }
  }'
```

(Swap `"image": "python:3.12-slim"` with `"image": "rocker/tidyverse:4.4.1"` if you want an interactive R debugging session!)

---

6. Housekeeping: Cleaning Up Finished Jobs

Completed jobs (`STATUS: Completed`) do not use CPU or GPU, but keeping dozens of old failed/completed jobs clutters `kubectl get pods`. Once you have verified your outputs are safely in `peg-bucket`:

```
# Delete a specific job (this also deletes its pod and logs):
kubectl delete job <job-name> -n stenc-ns

# Delete all successfully completed jobs in stenc-ns at once:
kubectl delete jobs --field-selector status.successful=1 -n stenc-ns
```